

第三次作业

贺子旭 物理学英才班 1701 U201710150

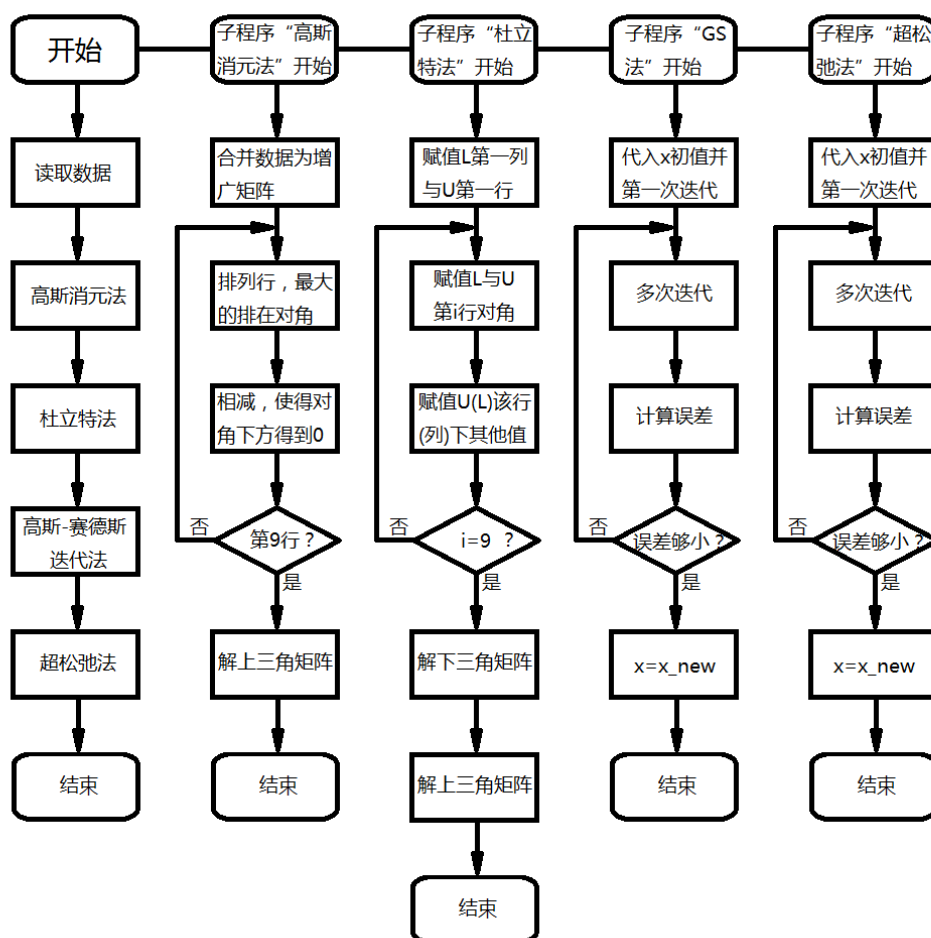
一、问题描述

利用高斯消元法、杜立特分解法、高斯-赛德尔迭代法、超松弛迭代法进行对线性方程组 $Ax=b$ 的求解

二、公式

1. 高斯消元法: $row_i = row_i + row_j \times \frac{-a_{ik}^{(k)}}{a_{kk}^{(k)}}$, $x_i = \frac{b_i - \sum_{j=i+1}^n a_{ij} x_j}{a_{ii}}$
2. 杜立特分解法: $A=LU$, $u_{ij} = a_{ij} - \sum_{k=1}^{i-1} l_{ik} u_{kj}$, $l_{ji} = \frac{a_{ji} - \sum_{k=1}^{j-1} l_{jk} u_{ki}}{u_{kk}}$, $y_i = b_i - \sum_{j=1}^{i-1} l_{ij} y_j$, $x_i = \frac{y_i - \sum_{j=i+1}^n a_{ij} x_j}{a_{ii}}$
3. 高斯-赛德尔迭代法: $x_i^{(k+1)} = \frac{-1}{a_{ii}} (\sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} + \sum_{j=i+1}^n a_{ij} x_j^{(k)} - b_i)$
4. 超松弛迭代法: $x_i^{(k+1)} = (1-\omega)x_i^{(k)} + \omega \frac{-1}{a_{ii}} (\sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} + \sum_{j=i+1}^n a_{ij} x_j^{(k)} - b_i)$

三、主要流程图



四、主要源代码

```
program H3
  implicit none
  real(kind=8) :: A_origine(9,9),b_origine(9)
  real(kind=8) :: x_Gauss(9),x_Doolittle(9),x_GS(9),x_Overrelaxation(9)
  real(kind=8) :: error

  interface
    subroutine readin(A,b)
      implicit none
      real(kind=8) :: A(9,9),b(9)
    end subroutine
    subroutine Gaussial_elimination(A,x,b)
      implicit none
      real(kind=8) :: A(9,9),b(9),x(9)
    end subroutine
    subroutine Doolittle_decomposition(A,x,b)
      implicit none
      real(kind=8) :: A(9,9),b(9),x(9)
    end subroutine
    subroutine Gauss_Seidel(A,x,b,error)
      implicit none
      real(kind=8) :: A(9,9),b(9),x(9),error
    end subroutine
    subroutine Overrelaxation(A,x,b,error)
      implicit none
      real(kind=8) :: A(9,9),x(9),b(9),error
    end subroutine
  end interface

  write(*,*) "本程序用于运用不同的方法计算同一个线性方程组Ax=b"
  write(*,*) "其对应A存放于Matrix.txt文件, b存放于Ax.txt文件"
  write(*,*) "为检查其结果正确性, 请打开Answer check.txt文件进行检查"
  write(*,*) " "

  write(*,*) "现在读取相应数据"
  call readin(A_origine,b_origine)           !读取
  write(*,*) "读取成功"
  write(*,*) " "

  write(*,*) "现在通过消元法与分解法解这个方程组, 理论上可以得到的精确解"
  write(*,*) " "

  call Gaussian_elimination(A_origine,x_Gauss,b_origine)
  write(*,*) "高斯消元法的结果是: "
  write(*,*) x_Gauss
  write(*,*) " "
  pause

  call Doolittle_decomposition(A_origine,x_Doolittle,b_origine)
  write(*,*) "杜立特分解法的结果是: "
  write(*,*) x_Doolittle
  write(*,*) " "
  pause

  write(*,*) " "
  write(*,*) "现在通过迭代法解这个方程组, 理论上不能得到的精确解"
  write(*,*) "一下两种迭代法迭法统一从原点开始迭代"
  write(*,*) "请输入预计误差: (误差请不要设置小于 $1.1 \times 10^{-16}$ ) 否则超松弛法可能会陷入死循环"
  read(*,*) error
  write(*,*) " "
  pause

  call Gauss_Seidel(A_origine,x_GS,b_origine,error)
  write(*,*) "高斯-赛德尔迭代法的结果是: "
  write(*,*) x_GS
  write(*,*) " "
  write(*,*) "以下为此次计算与真实值的偏差: "
  write(*,*) x_GS-x_Doolittle
  write(*,*) " "
  write(*,*) "该偏差在迭代次数达到54次时达到最小值, 此时error为 $10^{(-45)}$ 量级"
  write(*,*) "矩阵法迭代次数修正到57次, 且得到的最终结果偏差比直接计算更大。"
  write(*,*) " "
  pause
  write(*,*) " "

  call Overrelaxation(A_origine,x_Overrelaxation,b_origine,error)
  write(*,*) "超松弛法的结果是: "
  write(*,*) x_Overrelaxation
  write(*,*) " "
  write(*,*) "以下为此次计算与真实值的偏差: "
  write(*,*) x_Overrelaxation-x_Doolittle
  write(*,*) " "
  write(*,*) "该偏差在迭代次数达到67次时达到最小值, 此时error为 $10^{(-16)}$ 量级"
  pause
  stop
end program

include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Methord\Gaussian_elimination.F90"
include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Methord\Doolittle_decomposition.F90"
include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Methord\Gauss-Seidel.F90"
include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Methord\Overrelaxation.F90"

include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Assistant\Readin.F90"
include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Assistant\Triangle_solve.F90"
include "C:\Program Files\Microsoft Visual Studio\My code\homework3\Assistant\DLU.F90"
```

主程序

```

!本子程序利用高斯消元法解方程 $Ax=b$ 
!该方法致力于通过行间运算将增广矩阵上三角化，再解上三角矩阵的方程来解决这个问题

!该方法在理论上可以得到一个方程组的精确解
subroutine Gaussian_elimination(A,x,b)
  implicit none
  real(kind=8) :: A(9,9),b(9),x(9),times
  real(kind=8) :: solve(9,10),exchanger,minus
  integer(kind=1) :: counter(3)

  interface
    subroutine up_right_solve(solve,x)
      implicit none
      real(kind=8) :: solve(9,10),x(9)
    end subroutine
  end interface

  counter=1

  !对增广矩阵赋初值
  do while(counter(1)<=9)
    solve(counter(1),1:9)=A(counter(1),1:9)
    solve(counter(1),10)=b(counter(1))
    counter(1)=counter(1)+1
  end do
  counter(1)=1

  !处理矩阵，利用行间的加减运算将矩阵对角化
  do while(counter(1)<=9)

    !使用不完整的排序算法（由下到上一回）将最大的摆到最前面来
    counter(2)=9
    do while(counter(2)>counter(1))
      if(abs(solve(counter(2)-1,counter(1)))<abs(solve(counter(2),counter(1)))) then
        counter(3)=counter(1)
        do while(counter(3)<=10)
          exchanger=solve(counter(2)-1,counter(3))
          solve(counter(2)-1,counter(3))=solve(counter(2),counter(3))
          solve(counter(2),counter(3))=exchanger
          counter(3)=counter(3)+1
        end do
        counter(3)=counter(1)
      end if
      counter(2)=counter(2)-1
    end do
    counter(2)=counter(1)+1

    !进行减法，将矩阵转化为上三角矩阵
    do while(counter(2)<=9)
      times=solve(counter(2),counter(1))/solve(counter(1),counter(1))
      counter(3)=counter(1)
      do while(counter(3)<=10)
        solve(counter(2),counter(3))=solve(counter(2),counter(3))-solve(counter(1),counter(3))*times
        counter(3)=counter(3)+1
      end do
      counter(3)=counter(1)+1
      counter(2)=counter(2)+1
    end do
    counter(1)=counter(1)+1
  end do

  !函数：解上三角矩阵
  call up_right_solve(solve,x)

  return
end subroutine

```

高斯消元法

!本子程序使用杜立特法进行线性方程组 $Ax=b$ 的求解
 !该方法将 A 分解为一个下三角矩阵和一个上三角矩阵的乘积 $A=LU$ ，再通过解一个下三角方程 $Ly=b$ 和一个上三角方程 $Ux=y$ 来进行运算
 !该方法在理论上可以得到一个方程组的精确解

```

subroutine Doolittle_decomposition(A,x,b)
    implicit none
    real(kind=8) :: A(9,9),x(9),b(9),L(9,9),U(9,9),solve(9,10),y(9)
    integer(kind=1) :: counter(3)

    interface
        subroutine down_left_solve(solve,x)
            implicit none
            real(kind=8) :: solve(9,10),x(9)
        end subroutine
        subroutine up_right_solve(solve,x)
            implicit none
            real(kind=8) :: solve(9,10),x(9)
        end subroutine
    end interface

    U=0
    L=0
    counter=1

    !赋L的对角值和初始位置值（总是在初始位置出现程序错误，我养成习惯先定初值，顺便赋值L对角）
    do while(counter(1)<=9)
        L(counter(1),counter(1))=1
        U(1,counter(1))=A(1,counter(1))
        L(counter(1),1)=A(counter(1),1)/U(1,1)
        counter(1)=counter(1)+1
    end do

    !分解A为LU
    counter(1)=2
    do while(counter(1)<=9)
        counter(3)=1

        !U的对角值进行赋予（防止出错）
        U(counter(1),counter(1))=A(counter(1),counter(1))
        do while(counter(3)<=counter(1)-1)
            U(counter(1),counter(3))=U(counter(1),counter(3))-L(counter(1),counter(3))*U(counter(3),counter(1))
            counter(3)=counter(3)+1
        end do

        !对U和L分别进行赋值
        counter(2)=counter(1)+1
        do while(counter(2)<=9)
            counter(3)=1
            U(counter(1),counter(2))=A(counter(1),counter(2))
            L(counter(2),counter(1))=A(counter(2),counter(1))
            do while(counter(3)<=counter(1)-1)
                U(counter(1),counter(2))=U(counter(1),counter(2))-L(counter(1),counter(3))*U(counter(3),counter(2))
                L(counter(2),counter(1))=L(counter(2),counter(1))-L(counter(2),counter(3))*U(counter(3),counter(1))
                counter(3)=counter(3)+1
            end do
            L(counter(2),counter(1))=L(counter(2),counter(1))/U(counter(1),counter(1))
            counter(2)=counter(2)+1
        end do
        counter(1)=counter(1)+1
    end do

    !对Ly=b进行增广矩阵的赋值（解上下三角矩阵传参需要，个人设置）
    counter=1
    do while(counter(1)<=9)
        counter(2)=1
        do while(counter(2)<=9)
            solve(counter(1),counter(2))=L(counter(1),counter(2))
            counter(2)=counter(2)+1
        end do
        solve(counter(1),10)=b(counter(1))
        counter(1)=counter(1)+1
    end do

    !解下三角
    call down_left_solve(solve,y)

    !对Ux=y进行增广矩阵赋值（解上下三角矩阵传参需要，个人设置）
    counter=1
    do while(counter(1)<=9)
        counter(2)=1
        do while(counter(2)<=9)
            solve(counter(1),counter(2))=U(counter(1),counter(2))
            counter(2)=counter(2)+1
        end do
        solve(counter(1),10)=y(counter(1))
        counter(1)=counter(1)+1
    end do

    !解上三角
    call up_right_solve(solve,x)

    return
end subroutine
    
```

杜立特法

*本子程序用于利用超松弛迭代法进行方程的求根
 *该方法迭代时, $x_{new} = (1-\omega) * x_{old} + \omega * x_{new_GS}$, 只需在算出GS迭代法的 x_{new} 每一项后对其进行加权计算
 *个人尝试做了其对应的矩阵方法, 但出现了明显漏洞, 无法找到, 故没有继续尝试
 *该方法理论上不能得到精确解, 故将其给出的解与杜立特法结果(与高斯法结果完全一致)对比可以得到误差

```

subroutine Overrelaxation(A,x,b,error)
  implicit none
  real(kind=8) :: A(9,9),b(9),x(9)
  real(kind=8) :: D(9,9),L(9,9),U(9,9)
  real(kind=8) :: G_matrix(9,9),g_vector(9)
  real(kind=8) :: error,delta,omega
  real(kind=8) :: x_new(9),x_old(9)
  integer(kind=1) :: counter(3)
  interface
    subroutine DLU(A,D,L,U)
      implicit none
      real(kind=8) :: A(9,9),D(9,9),L(9,9),U(9,9)
    end subroutine
  end interface

  omega=1.5d0
  counter=1
  G_matrix=0
  g_vector=0

  call DLU(A,D,L,U)

  *对迭代矩阵和向量进行赋值
  do while(counter(1)<=9)
    counter(2)=1
    g_vector(counter(1))=b(counter(1))/D(counter(1),counter(1))
    do while(counter(2)<=9)
      G_matrix(counter(1),counter(2))=(L(counter(1),counter(2))+U(counter(1),counter(2)))/D(counter(1),counter(1))
      counter(2)=counter(2)+1
    end do
    counter(1)=counter(1)+1
  end do

  *进行赋值与初次迭代
  counter=1
  x_new=0
  x_old=x_new
  do while(counter(1)<=9)
    counter(2)=1
    x_new(counter(1))=0
    do while(counter(2)<=9)
      x_new(counter(1))=x_new(counter(1))+G_matrix(counter(1),counter(2))*x_new(counter(2))
      counter(2)=counter(2)+1
    end do
    x_new(counter(1))=x_new(counter(1))+g_vector(counter(1))
    counter(1)=counter(1)+1
  end do
  x_new=x_new*omega+x_old*(1d0-omega)
  delta=0
  do while(counter(3)<=9)
    delta=delta+(x_old(counter(3))-x_new(counter(3)))**2
    counter(3)=counter(3)+1
  end do
  delta=sqrt(delta)
  counter=1

  *进行迭代
  do while(delta>=error)
    x_old=x_new

    *迭代过程
    do while(counter(1)<=9)
      counter(2)=1
      x_new(counter(1))=0
      do while(counter(2)<=9)
        x_new(counter(1))=x_new(counter(1))+G_matrix(counter(1),counter(2))*x_new(counter(2))
        counter(2)=counter(2)+1
      end do
      x_new(counter(1))=x_new(counter(1))+g_vector(counter(1))
      counter(1)=counter(1)+1
    end do
    x_new=x_new*omega+x_old*(1d0-omega)
    counter(1)=1
    counter(2)=1

    *改变量计算
    delta=0
    do while(counter(1)<=9)
      delta=delta+(x_old(counter(1))-x_new(counter(1)))**2
      counter(1)=counter(1)+1
    end do
    delta=sqrt(delta)
    counter(1)=1
    counter(3)=counter(3)+1
  end do
  x=x_new
  write(*, "('一共经历了',I3,'次迭代')") counter(3)

  return
end subroutine
  
```

超松弛法

*本子程序将方阵A分解为一个上三角矩阵-U和对角矩阵D以及下三角矩阵-L的和, 即 $A=D-U-L$
 *D、L、U这三个参数在迭代法计算中很重要

```
subroutine DLU(A,D,L,U)
  implicit none
  real(kind=8) :: A(9,9),D(9,9),L(9,9),U(9,9)
  integer(kind=1) :: counter(2)

  counter=1
  D=0
  L=0
  U=0
  do while (counter(1)<=9)
    D(counter(1),counter(1))=A(counter(1),counter(1))
    counter(2)=counter(1)+1
    do while (counter(2)<=9)
      U(counter(1),counter(2))=A(counter(1),counter(2))
      L(counter(2),counter(1))=A(counter(2),counter(1))
      counter(2)=counter(2)+1
    end do
    counter(1)=counter(1)+1
  end do
  L=-1d0*L
  U=-1d0*U
  return
end subroutine
```

*解上三角矩阵方程

```
subroutine up_right_solve(solve,x)
  implicit none
  real(kind=8) :: solve(9,10),x(9)
  integer(kind=1) :: counter(2)

  counter=9
  do while(counter(1)>=1)
    counter(2)=9
    x(counter(1))=solve(counter(1),10)
    do while(counter(2)>counter(1))
      x(counter(1))=x(counter(1))-solve(counter(1),counter(2))*x(counter(2))
      counter(2)=counter(2)-1
    end do
    x(counter(1))=x(counter(1))/solve(counter(1),counter(1))
    counter(1)=counter(1)-1
  end do

  return
end subroutine
```

*解下三角矩阵方程

```
subroutine down_left_solve(solve,x)
  implicit none
  real(kind=8) :: solve(9,10),x(9)
  integer(kind=1) :: counter(2)

  counter=1
  do while(counter(1)<=9)
    x(counter(1))=solve(counter(1),10)
    counter(2)=1
    do while(counter(2)<counter(1))
      x(counter(1))=x(counter(1))-solve(counter(1),counter(2))*x(counter(2))
      counter(2)=counter(2)+1
    end do
    x(counter(1))=x(counter(1))/solve(counter(1),counter(1))
    counter(1)=counter(1)+1
  end do
```

五、运行结果

```
C:\Program Files\Microsoft Visual Studio\My code\homework3\Main\Debug\Prob.exe
本程序用于运用不同的方法计算同一个线性方程组Ax=b
其对应A存放于Matrix.txt文件，b存放于Ax.txt文件
为检查其结果正确性，请打开Answer check.txt文件进行检查

现在读取相应数据
读取成功

现在通过消元法与分解法解这个方程组，理论上可以得到的精确解

高斯消元法的结果是：
-0.289233816015754      0.345435715779115      -0.712811731086879
-0.220608510570529      -0.430400432704022      0.154308739838311
-5.782287328904062E-002  0.201053894823681      0.290228661879745

Fortran Pause - Enter command<CR> or <CR> to continue.

杜立特分解法的结果是：
-0.289233816015754      0.345435715779115      -0.712811731086879
-0.220608510570529      -0.430400432704022      0.154308739838311
-5.782287328904062E-002  0.201053894823681      0.290228661879745

Fortran Pause - Enter command<CR> or <CR> to continue.

现在通过迭代法解这个方程组，理论上不能得到的精确解
一下两种迭代法迭代统一从原点开始迭代
请输入预计误差：（误差请不要设置小于 $1.1 \times 10^{-16}$  否则超松弛法可能会陷入死循环）
0.0000000000000001

Fortran Pause - Enter command<CR> or <CR> to continue.

一共经历了 50次迭代
高斯-赛德尔迭代法的结果是：
-0.289233816015754      0.345435715779116      -0.712811731086879
-0.220608510570528      -0.430400432704022      0.154308739838311
-5.782287328904050E-002  0.201053894823681      0.290228661879745

以下为此次计算与真实值的偏差：
2.775557561562891E-016  1.110223024625157E-016  1.110223024625157E-016
1.387778780781446E-016  1.110223024625157E-016  1.665334536937735E-016
1.249000902703301E-016  0.000000000000000E+000  1.110223024625157E-016

该偏差在迭代次数达到54次时达到最小值，此时error为 $10^{-45}$ 量级
矩阵法迭代次数修正到57次，且得到的最终结果偏差比直接计算更大。

Fortran Pause - Enter command<CR> or <CR> to continue.

一共经历了 62次迭代
超松弛法的结果是：
-0.289233816015755      0.345435715779115      -0.712811731086879
-0.220608510570529      -0.430400432704022      0.154308739838311
-5.782287328904066E-002  0.201053894823681      0.290228661879745

以下为此次计算与真实值的偏差：
-3.885780586188048E-016 -5.551115123125783E-017  0.000000000000000E+000
2.775557561562891E-017  0.000000000000000E+000 -5.551115123125783E-017
-3.469446951953614E-017  0.000000000000000E+000  5.551115123125783E-017

该偏差在迭代次数达到67次时达到最小值，此时error为 $10^{-16}$ 量级
Fortran Pause - Enter command<CR> or <CR> to continue.
```


六、总结

1. 利用在多重循环里面用循环算求和，要在合适的位置进行“归零”
2. 仔细阅读，利用好存储特性进行迭代
3. 利用好系统自带的库函数
4. 优化建议：传参时可以删除不必要参数而利用与它等值的参数传递